AI 101 — build it, ship it, log it — then your instrument

Lecture 1 • Fri 11 Sep 2026 • 14:20–16:20 • R311 IAMS

Two promises for today:
a page that teaches one idea, on the public web, **before the break** •
your own phone app measuring your ESP32 **by the end**

What we build together

One class instrument — an ESP32-S3 dev board on a class board with **five blocks**, one per student:

- **B1** precision inputs (8 × ±10 V, 16-bit)
- **B2** power & rails
- **B3** signal generation (2 × ±10 V out)
- **B4** isolated switching (4 relays, 2 optos)
- **B5** digital I/O & timing (5 V TTL, TRIG)

One repo `class-board-2026`, **one phone app**, **one Python client**.

You read your block, route it, write its driver and demo it.

The board is ordered **Mon 28 Sep** from everyone's pull requests. Boards back ≈ 12 Oct. Demos week of 26 Oct.

Why we start away from the project

AI-assisted work is a **general skill**. The same three moves serve a simulation, a website, a video, a report, a KiCad board, a firmware driver:

Build — from a written spec
Ship — where others can see it
Log — what you did, in plain text

Today, twice:

1. **teach one idea you know** — EM, electronics, mechanics, PID — to a high-school student: a page, a simulation, a game. The answer is known before the code exists.
2. the **ESP32 in your hand** — a phone app that measures something. The answer is not known yet.

Electronics design is *one* application of AI, not the syllabus.

The one-sentence thesis

> An AI agent is a **junior engineer with no memory**.
> It is exactly as good as the project documentation, the task specification and the verification you give it.

Everything today follows from that sentence.

The five working practices

1. **Spec first.** Requirements live in `SPEC.md`, not in chat scrollback. Ask for a plan before code.
2. **Teach the repo, not the session.** `CLAUDE.md`: conventions, pinout, build/flash commands, how to test. Written once, loaded every session.
3. **Small, fresh sessions.** One task = one session. Finish, commit, start clean. Long sessions rot.
4. **Handover notes.** `PROGRESS.md` at the end of every session: done / verified / next / gotchas.
5. **AI writes, you verify.** No verification plan → not yours to ship.

Markdown is your lab notebook

`PROGRESS.md`, `SPEC.md`, `notes.md`, `README.md`, this deck, the workbook — all plain text.

- it **diffs** and lives in git next to the code
- it is **searchable** and readable on a phone, on GitHub, in ten years
- the AI **reads it first** at the start of the next session
- the lingua franca of AI work: specs, logs, slides, reports

```
## 2026-09-11 — session 1
- Done: rc-filter teaching page from SPEC.md; deployed to https://rc-filter-xxxx.pages.dev
- Verified: f_c 1591 Hz (pred. 1591.5), −3.0 dB / −45° there; sliders work on my phone
- Next: the teaching questions grade the answer; RLC as idea two
- Gotchas: Cloudflare output directory must be "/"
```

"Verify" means a number or an observation

Today	Later
Is f_c where **$1/(2\pi RC)$** said it would be?	Does DRC say **0**?
Does sd fall as **$1/\sqrt{N}$** on your phone — and if not, why?	Does the scope show **± 5.00 V**?

"It looks right" is not verification. "It works" is not verification.
Write the number down. That is the whole trick.

The loop in miniature — today's two exercises

E1a • Build

your topic, your repo • five-line `SPEC.md` — learner outcome + a number predicted on paper • plan • one `index.html` • the check • commit

E1b • Ship + log

push • Cloudflare Pages • URL on your phone • `PROGRESS.md` • push → redeploys itself

E2 • Your phone app measures your ESP32

one command out • mean + sd back • flash • sd vs N against $1/\sqrt{N}$ — explain the difference • branch • PR with the numbers

Nobody leaves without it.

E1a — build: teach one idea to a high-school student

Pick a topic you know — EM • electronics • classical mechanics • PID • your research. Build the thing that teaches it: a page with sliders and a live plot, a simulation, a game.

```
gh repo create <you>/rc-filter --public --clone && cd rc-filter && code .
```

You write `SPEC.md` — five lines: what the learner can do afterwards • what it shows • **the check: a number predicted on paper** (1 kΩ, 100 nF → f_c = 1591.5 Hz, −3 dB, −45°) • a teaching check (a question with a hidden answer) • one file, canvas + JS, no framework, no build, works on a phone.

Then: *"Read SPEC.md. Plan in five bullets. Then write index.html."* Reject a plan that draws a picture instead of computing the physics. Open it. **Read your number off the page.** Prediction vs result into `SPEC.md`. Commit.

Topics with a check: RC filter (f_c) • PID (settling time from the poles) • two charges (field on the axis) • cart-pole (period ×

E1b — ship + log

1. `git push -u origin main`
2. Cloudflare → **Workers & Pages → Create → Pages → Connect to Git** → your repo → build command *blank*, output `/` → Deploy
3. Open `https://rc-filter-xxxx.pages.dev` **on your phone**. Move a slider. Send it to a non-physicist.
4. **You** write `PROGRESS.md`: done / verified / next / gotchas. Commit, push — and watch it redeploy itself.

Fallback: GitHub → Settings → Pages → branch main. That is continuous deployment; you just did it. This repo is your course page from now on.

Break

When we come back: join your board's WiFi.

The instrument on your phone

Your board is an **access point**: LED blue → phone joins `instrument-XXXX` (password `instrument`) → **http://192.168.4.1** → tap *Red*.

firmware on the ESP32-S3
↑↓ one **WebSocket**
phone app (a web page the board serves)
↑↓ same messages
`instrument.py` in Python on a PC

phone → board, one JSON message:
`{"block":"base","cmd":"led","args":{"r":255,"g":0,"b":0}}`
board → phone, one reply
board → everyone, **20 × per second**:
status numbers → the chart, the alarms

The two files a control touches

`firmware/src/blocks/base/BaseBlock.cpp`

```
if (strcmp(c, "led") == 0) {
  r_ = a["r"] | r_;  g_ = a["g"] | g_;  b_ = a["b"] | b_;
  showLed();
  reply["r"] = r_; ... return true;
}
// status(): out["counter"] = counter_;
```

`host/pwa/panels/base.js`

```
btn.onclick = () =>
  api.send('base', 'led', { r, g, b });
// onStatus(st):
els.counter.textContent = st.counter;
```

A command is **one `if` and one button**. A number is **one `out[...]` and one `<span>`**.

E2 — your phone app measures something on your ESP32

One command out, one live measurement back — with its noise. Default recipe (workbook § A.4):

1. `BaseBlock.h` : a 1000-sample ring buffer, `adcN_ = 16` , `adcV_` , `adcSd_`
2. `loop()` : every 1000 µs `analogReadMilliVolts(4)` into the ring; mean and sd over the last N — **no** `delay()`
3. `handle()` : `set_avg {n}` (1…1000) • `status()` : `adc_v` , `adc_sd` , `avg_n`
4. `panels/base.js` : an **N** input + *Set* → `api.send('base','set_avg',{n})` ; two readouts filled in `onStatus`
5. `pio run -e esp32s3-sim -t upload && pio run -e esp32s3-sim -t uploadfs`
6. **Verify against a prediction:** sd for N = 1, 4, 16, 64, 256. White noise says $1/\sqrt{N}$. It won't obey exactly — quantisation floor? mains pickup on the wire? **Say why.**

The class board

Block	What	Parts
B1	8 × ±10 V 16-bit inputs, ADS8688	≈ 32
B2	USB-C / 5 V jack → +5V, +3V3, ±12 V, +5VA	≈ 20
B3	2 × ±10 V outputs, DAC8563 + OPA2192	≈ 15
B4	4 relays, 2 isolated inputs	≈ 24
B5	8 × 5 V TTL out, TRIG in/out	≈ 15

Front panel, **3 × 4 SMA at 20 mm**:

```
AO1 AO2 TRIG AUX
AI1 AI2 AI3 AI4
AI5 AI6 AI7 AI8
```

Base (instructor): dev-board socket, buses, LED, Qwiic, expansion header. JLC assembles everything; you solder nothing.

Your block — volunteers first, then lots

Five names, five blocks. From now on:

- you **read** it (two questions and one design number this week)
- you **route** it (week 2, your zone of the PCB)
- you **write its driver and panel** (week 3, in sim)
- you **measure one number** on it and **demo** it (Oct)

The tutor writes your block into `PROGRESS.md` and opens your block page.

Beyond the baseline

The **hardware is the shared baseline** — fixed by budget and timeline; JLC builds it. The **software, firmware and app are open.** That is where you show what you can do.

Whenever you are ahead, the question is: *what problem in my lab could this instrument solve?*

- a **data logger** to CSV that a tool of yours reads · **LINE / Telegram alerts** from the alarm engine
- **remote access** + a **Python sweep** overnight · a **calibration routine** on the board
- a **PID / controller block** — today's PID page, made real · a **second node** (S3-CAM on a gauge)

Bring a real problem to Lecture 2; the tutor helps you write its `SPEC.md`. Optional fifth item at the demo.

Homework 1

- [ ] **E2 finish + merge** — instructor reviews Mon; address one comment; merge
- [ ] **E3 read your block** — V3, two answers **and the design number** in `notes.md`
- [ ] **E4 KiCad ready** — V4, install KiCad 10 (the library is in the repo), screenshot your zone
- [ ] **E5 your SPEC paragraph** — what, **the number you will measure**, how you show it
- [ ] **reading** — workbook ch. C (git, KiCad); ch. A where needed

`/tutor HW1` paces it. LINE group for help. **Office hour Wed 16:00** — the flashing safety net.

Every work session ends with `PROGRESS.md` + commit — your day-1 repo and the class repo alike.

Next Friday: your part of the board

Electronics 101 on **our** schematic · KiCad's six operations · place your missing parts · route your zone · your first PR into the class board

Before then: HW1, KiCad 10 installed, the project open.